

Making Fault Localization in Online Service Systems More Actionable and Interpretable

KE XV and SHIKAI GUO, Dalian Maritime University, Dalian, China

HUI LI, Information Science and Technology College, Dalian Maritime University, Dalian, China

CHENCHEN LI, Liaoning Normal University, Dalian, China

RONG CHEN, Dalian Maritime University, Dalian, China

XIAOCHEN LI, School of Software, Dalian University of Technology, Dalian, China

HE JIANG, School of Software, Dalian University of Technology, Dalian, China

Online service systems struggle with accurately and quickly pinpointing and resolving failures within their intricate systems, and it therefore emerges the solutions for fault localization in the code. However, the previous fault localization models suffer from low localization accuracy and poor interpretability due to the complex dependencies among fault characteristics in industrial practice. To address this issue, challenges brought by the long-distance dependencies among fault features and the unbalanced distribution of fault knowledge, and to improve the interpretability of the model, we present a fault localization model in online service systems more actionable and interpretable, named FL-AIer. Specifically, FL-AIer consists of two components: the feature encoding component and the fault localization component. The feature encoding component utilizes graph attention networks to capture the complex spatio-temporal dependencies within fault features. Then, the fault localization component adopts a three-stage approach, leveraging a multi-attention mechanism to identify and prioritize the most relevant fault features for precise localization. Additionally, the Fault Knowledge Balancing module it contains introduces a weighted Kullback-Leibler divergence loss function to ensure that the model pays adequate attention to all fault features, addressing the issue of imbalanced fault knowledge distribution and enhancing localization performance. We conducted extensive experiments on four datasets, and the results demonstrated that FL-AIer effectively addressing the challenges of fault localization in online system environments, and consistently outperforms the state-of-the-art methods across various evaluation metrics such as A@1, A@2, A@3, A@5, and MAR. For instance, FL-AIer achieves significant improvements of 5.82%, 10.77%, 4.20%, and 15.56% on the A@1 metric, respectively. These results fully demonstrate the excellent effectiveness of FL-AIer in effectively addressing the challenges of fault localization in online system environments, surpassing the performance of existing state-of-the-art methods.

This work was supported by the National Natural Science Foundation of China (Nos. 62472062, 62202079), the Dalian Science and Technology Innovation Fund Project (No. 2024JJ12GX022), the Dalian Excellent Young Project (No. 2022RY35), and the Fundamental Research Funds for the Central Universities (No. 3132024257).

Authors' Contact Information: Ke Xv, Dalian Maritime University, Dalian, China; e-mail: xvke@dlmu.edu.cn; Shikai Guo (corresponding author), Dalian Maritime University, Dalian, China; e-mail: shikai.guo@dlmu.edu.cn; Hui Li, Information Science and Technology College, Dalian Maritime University, Dalian, China; e-mail: li_hui@dlmu.edu.cn; Chenchen Li, Liaoning Normal University, Dalian, China; e-mail: lcc@lnnu.edu.cn; Rong Chen, Dalian Maritime University, Dalian, China; e-mail: rchen@dlmu.edu.cn; Xiaochen Li, School of Software, Dalian University of Technology, Dalian, China; e-mail: xiaochen.li@dlut.edu.cn; He Jiang, School of Software, Dalian University of Technology, Dalian, China; e-mail: jianghe@dlut.edu.cn.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM 1557-7392/2025/7-ART165

<https://doi.org/10.1145/3714466>

CCS Concepts: • **Software and its engineering** → **Software creation and management**; **Collaboration in software development**; **Open source model**;

Additional Key Words and Phrases: Fault Localization, Online Service Systems, Fault Indicator Dependence

ACM Reference format:

Ke Xv, Shikai Guo, Hui Li, Chenchen Li, Rong Chen, Xiaochen Li, and He Jiang. 2025. Making Fault Localization in Online Service Systems More Actionable and Interpretable. *ACM Trans. Softw. Eng. Methodol.* 34, 6, Article 165 (July 2025), 26 pages.

<https://doi.org/10.1145/3714466>

1 Introduction

Currently, online service systems have gradually replaced traditional software systems and have permeated various aspects of real-world, playing an increasingly irreplaceable role [1–4]. E-commerce giants, such as Amazon and Taobao, offer convenient online shopping experiences. Through online banking and payment applications, such as Alipay and WeChat Pay, users can manage their bank accounts, transfer funds, and pay bills anytime and anywhere. Social media platforms, such as Facebook and WeChat, have become the primary means for people to share their lives, exchange opinions, and establish social networks.

Despite significant efforts made in maintaining software services, such as closely monitoring various metrics like average response time or memory usage on a 24-hour basis [3, 5], failures are still inevitable due to the scale and complexity of software services. These failures can arise from various causes, including code defects, hardware failures, network issues, or improper user operations. They can occur suddenly, leading to system performance degradation, data loss, or service interruption, thereby having a severe impact on user experience and the economic benefits of enterprises [6, 7].

- On the evening of 5 March 2023, at approximately 20:20, users of the popular video-sharing platform Bilibili encountered significant difficulties accessing the “Anime” and “Cinema” video detail pages across both mobile and desktop platforms. This outage persisted for approximately an hour, coinciding with peak user activity times.
- Vipshop’s 5 June 2023 statement, “Handling of March 29 Data Center Outage,” reported a cooling system failure at Nansha IDC from 00:14 to 12:01. This incident led to a complete shutdown of Vipshop’s online services for a duration of 12 hours, resulting in substantial financial losses amounting to over 100 million yuan. Furthermore, the outage impacted approximately 8 million customers. The incident was categorized as a P0-level fault, indicating its severity and urgency for resolution [8].
- During the period spanning the evening of 27 November to the morning of 28 November 2023, DIDI, a prominent ride-hailing platform, experienced a widespread system outage lasting 12 hours nationwide. This outage resulted in significant network disruptions and led to the cancelation of numerous customer orders. Despite experiencing a remarkable 30.8% annual increase in orders, the outage inflicted substantial losses, estimated to be over 10 million orders, amounting to around 400 million yuan in financial damages. Additionally, the incident severely eroded customer trust in DIDI’s services and contributed to a notable decline in the company’s market share, dropping from 90% to approximately 70% [9].

Given the above issues, fault maintenance of online service systems is crucial, and the core of maintenance lies in accurately and quickly locating the fault occurrence sites. Since online service systems usually involve a large amount of data and complex operating environments, the use of

machine learning methods for fault location can reduce the time and effort required for manual analysis. These methods are widely adopted for dynamically adjusting as the system data changes, and adapting to the analysis and processing of complexity and large-scale data. However, the poor quality of training data caused by the complexity and interdependence of fault features poses significant challenges to machine learning training [10]. Additionally, the interpretability of the model is also questionable. Therefore, efficient, accurate, and well-interpretable fault location is urgently needed in the industrial sector. As a result, many previous methods have been presented for this purpose.

In previous studies, fault localization methods primarily focused on fault levels that are not directly actionable. The “Fingerprint” model, based on fault indicators, was first applied in large-scale industrial production environments [11] to determine the type of faults. However, this indicator-based method is too granular (for example, if only memory usage is localized, it is difficult to determine exactly what happened). Subsequently, Zhou et al. [12] proposed a model called MEPFL to address the issue of overly granular indicators, which bases fault localization on fault logs. Although this method focuses on the file level where faults occur, its granularity is too coarse to understand the specific type of fault that occurred, which remains challenging. To overcome the limitations of MEPFL, Li et al. [13] proposed an improved model named DéjàVu, which defines fault types (such as memory leaks) and the files where faults occur as a single fault unit. The goal of this model is to provide engineers with suspicious fault units, effectively saving time in fault localization. Compared to MEPFL, DéjàVu’s fault localization performance has significantly improved. However, despite ongoing progress in this field, the task of fault localization still retains inherent complexity. Two major challenges affect fault localization effectiveness:

Challenge 1. The dependencies caused by the complex logic between interactive code segments in online service systems present significant challenges for fault localization. A single fault unit has the potential to trigger anomalies not only within the same component but also across different metrics of various components. This interconnectivity complicates the identification and resolution of faults, as addressing the complex dependencies between fault characteristics becomes increasingly intricate.

Challenge 2. The frequency at which different types of faults occur can vary widely, further complicating the fault management process. When training models based on repeatedly occurring historical faults, there is a risk that the model may over-focus on the fault types that occur more frequently. Consequently, fault categories that occur less frequently may be overlooked or underrepresented in the training data, potentially leading to suboptimal performance in fault detection and resolution. Thus, addressing the imbalanced distribution of fault occurrences becomes paramount to ensuring the effectiveness and robustness of fault management systems in online service environments.

For Challenge 1: The Dependencies between Fault Features. Among the existing methods, many models such as **Multilayer Perceptrons (MLP)** can handle complex input data, but they struggle to model the deep and nonlinear dependencies between fault features. MLP performs feature transformation through a simple layered structure, ignoring the complex interactions that may exist between fault features, such as conditional dependencies and causal relationships. This results in poor performance when faced with complex fault patterns. Additionally, traditional methods often lack customized designs for specific fault scenarios, making it difficult to accurately capture fault features in specific domains. Current methods employ simple feature processing and fail to fully utilize domain knowledge to construct feature representations that better reflect the nature of faults. The lack of interactive modeling ignores the dynamic interactions and mutual influences between fault features, limiting the model’s predictive capabilities. Limited generalization ability challenges the model’s adaptability and generalization when faced with different industrial environments and

fault types. For Challenge 2: The Imbalance in Fault Data Distribution. The current state-of-the-art methods use oversampling techniques to balance datasets, but this approach may introduce the risk of overfitting and may not effectively enhance the model's ability to identify minority classes (i.e., fault classes). Simply increasing the number of minority class samples may lead the model to learn useless noise information, affecting generalization performance. Oversampling does not fully consider the intrinsic relationships and structures between data, making it difficult to fundamentally solve the problem of distribution imbalance.

To address these challenges, we propose a model named FL-AIer, which comprises two components: the feature encoding component, the fault localization component, and the Fault Knowledge Balancing module. Specifically, the feature encoding component encodes the temporal and spatial features of fault units based on multi-layer **Graph Attention Networks (GATs)** [14, 15]. Firstly, it captures the long-term dependencies of fault unit metrics over time. Then GAT is used to perform attention operations on fault units at each node, solving the mutual dependencies caused by multi-hop propagation of fault units. The fault localization component further addresses the issue of complex dependencies among fault features. This component is based on a multi-head attention mechanism, where each attention head learns different attention weights, focusing on the correlations between aggregated features [16]. Additionally, it includes a Fault Knowledge Balancing module that tackles the problem of unbalanced fault knowledge distribution. This module employs a weighted KL divergence loss function, which differentiates samples based on the frequency of fault recurrences and assigns different loss weights to different samples. The FL-AIer model focuses on samples that are more challenging to classify, thereby enhancing performance in handling imbalanced fault classification.

We compared FL-AIer with the state-of-the-art methods on four real projects. These projects include a microservice system from a major ISP, a service-oriented system from a commercial bank, an Oracle database from another commercial bank, and an open source microservice benchmark system [17]. The experimental results show that FL-AIer has improved over the state-of-the-art method DéjàVu on multiple evaluation metrics: A@1, A@2, A@3, A@5, and **Mean Average Rank (MAR)**. Taking A@1 as an example, FL-AIer achieved 81.67%, 73.34%, 64.42%, and 87.39% on four datasets from sys_A , sys_B , sys_C , and sys_D , respectively and improved DéjàVu method by up to 5.82%, 10.77%, 4.20%, and 15.56%. Overall, FL-AIer combines multiple advanced machine learning techniques to efficiently locate suspicious fault locations, providing an effective method for fault localization in online service systems.

The main contributions of our work are as follows:

- We propose a model named FL-AIer, which is trained offline using historical faults in a given online service system and provides fault localization for fault units online. FL-AIer integrates two components: a feature encoding component and a fault localization component effectively addressing the challenges of fault localization.
- Extensive experiments are conducted to evaluate the effectiveness of precise fault localization. Taking dataset sys_A as an example, FL-AIer achieved 81.67%, 93.76%, 95.88%, 98.25%, and 1.42 on A@1, A@2, A@3, A@5, and MAR metrics, respectively, representing improvements of 5.82%, 3.74%, 2.66%, 2.05%, and 16.9% compared to the state-of-the-art methods. The results show that it significantly outperforms state-of-the-art methods on multiple evaluation metrics, demonstrating its exceptional fault localization performance.
- We release the complete code of FL-AIer and the evaluation dataset to promote reproducibility and facilitate further investigation [18].

The remainder of this article is organized as follows. Section 2 discusses the background motivation for our work and reviews relevant literature in the field. Section 3 introduces the main

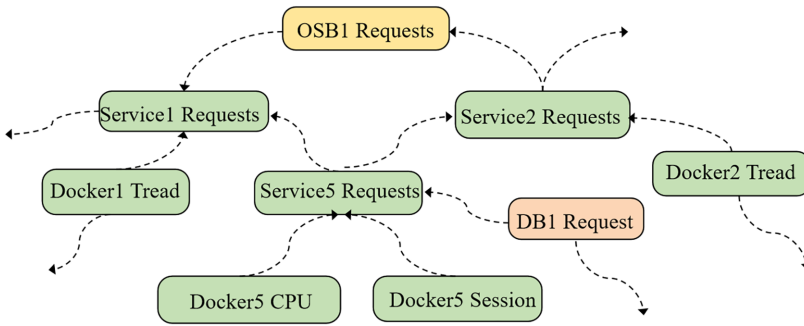


Fig. 1. Dependency relationships of fault units in sys_A [13].

components of FL-AIer. Section 4 analyzes the excellent interpretability of FL-AIer. Sections 5 and 6 present the experimental setup and corresponding results, respectively. Section 7 discusses effectiveness. Finally, Section 8 summarizes this study and outlines possible future directions.

2 Motivation and Related Work

In this section, we first illustrate our motivation with four examples in Section 2.1. Then, we enumerate the relevant works in the field in Section 2.2.

2.1 Motivating Examples

There are now many issues in fault location, as mentioned in the following content, which inspire us to continue exploring this field.

Prevalence of Recurring Faults. Li et al. [13] reported that in a major bank 300 applications, at least 60% of faults are recurring, averaging a 74.38% recurrence rate. Dogga et al. [19] reported that recurring categories of root causes led to 94% of faults in a major SaaS company and Lee and Iyer discovered that 70% of the faults they studied were recurring [20].

Complex Dependencies between Fault Units. A single unit's fault can impact other faults within the same component and spread to others, altering their fault indicators. In Figure 1, taken from a major ISP's microservice system dataset, a complex dependency graph with 27 components illustrates how a memory exhaustion fault in Docker5 affects various services. This fault initially impacts Service5, then propagates to OSB1 through its reliance on Service1 and Service2 [13].

Inefficient Industrial Fault Localization. Currently, fault localization in industry is mainly done manually. The common fault handling process typically involves receiving a fault alert, determining the type of fault (whether it is caused by software or hardware), locating the fault based on relevant log files and fault indicators, implementing measures to rectify the fault, and confirming the restoration of normal system operation. This process often involves multiple personnel from different departments and consumes a significant amount of diagnostic time. The study on fault records from a large commercial bank revealed that the average diagnostic time for faults in the company's online service systems is 28.98 minutes, with 25%, 50%, and 75% percentiles at 4.27/9.65/25.81 minutes, respectively. Additionally, the average time cost for engineers to locate root cause of the fault is 9.2 minutes, with 25%, 50%, and 75% percentiles at 2.8/5.0/10.0 minutes, respectively [13].

Shortcomings in the Interpretability of Previous Fault Prediction Algorithms. Most previous fault localization methods [7, 11, 17, 21–23] do not explicitly provide explanations. iSQUAD [24], Fingerprint [11], and Brandón [21] work by identifying similar historical failures. Many previous

explanation methods for deep learning models [25–27] focus on understanding the internal mechanisms of the model but do not provide reliable evidence for engineers to accurately determine the type of fault.

In summary, due to the widespread occurrence of recurrent faults resulting in significant losses, the current accuracy of fault diagnosis is not satisfactory. Manual fault localization in industrial settings is inefficient, there exist complex dependencies between fault characteristics and shortcomings in the interpretability of existing fault prediction algorithms. Therefore, our research is highly meaningful.

2.2 Related Work

In order to better apply machine learning in the field of fault localization, previous works have made many attempts in fault feature extraction, fault localization, and interpretability.

2.2.1 Feature Extraction. Fingerprint [11] and MonitorRank [28] take historical faults and fault indicator features derived from current monitoring as input, operating by finding similar historical faults to directly provide the type of fault occurrence. However, they only considered the fault characteristics in the time dimension when the fault occurs, such as changes in CPU usage, network latency, memory usage, and other indicators. To further improve the quality of extracted fault features, Brandón et al. [21] proposed a root cause analysis framework based on fault architecture diagrams. These diagrams can be used to compare any anomalies occurring in the system with an anomaly graph library used as a user troubleshooting knowledge base, introducing spatial features of faults. However, there is room for improvement in their integration of temporal and spatial features of faults. Additionally, they did not further process and screen the extracted features to select more effective ones for fault localization. In contrast, FL-AIer further combines these features and removes those with less significance for model classification. Additionally, some traditional machine learning methods are based on fault indicators for localization, but this localization granularity is too fine, making it difficult to determine the specific components where the fault occurs [29]. Besides metrics, many previous works have also extensively studied logs [29–32] and traces [12, 33, 34] used for fault localization.

2.2.2 Fault Localization. Most of the previous fault localization methods [35, 36] are unsupervised and heuristic, encompassing both multi-class and binary classification frameworks. Some prior works [40] localize root causes in an unsupervised manner, leveraging historical faults to adjust parameters. However, such approaches hinder the interpretability of the models. MEPFL [12] constructs and trains supervised machine learning models, which to a certain extent enhances the interpretability and credibility of the model. Nevertheless, it treats the localization of faulty microservices as a multi-class classification problem. This multi-class framework increases the complexity and number of parameters of the model, making it difficult to handle class imbalance issues and increasing computational costs. FluxRank [23], which also adopts a supervised approach for training, ranks suspicious metric summaries. These summaries are groups of similar metrics that improve the interpretability issues associated with unsupervised methods. However, it cannot directly provide the type of fault, and its results do not effectively address the goal of rapidly and accurately locating faults. To address the aforementioned issues, FL-AIer employs a binary classification framework and learns from historical faults. It defines the combination of fault indicators and the methods where faults occur as a fault unit. When a fault occurs in the online service system, FL-AIer directly provides engineers with suspicious scores for potential fault units, enhancing interpretability and reducing model complexity for rapid fault localization.

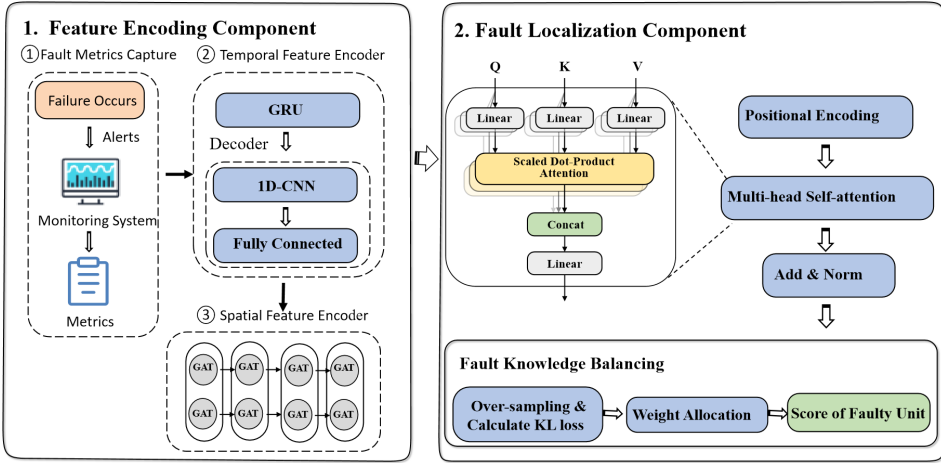


Fig. 2. The framework of the FL-Aler.

2.2.3 Interpretability. Currently, most fault localization methods, such as iSQUAD, Fingerprint, and Brandón, do not provide explicit interpretability methods. While some existing interpretability methods for deep learning focus on understanding the internal mechanisms of models, they do not meet our requirements [37]. Since the internal complex network mechanisms of deep learning often do not require engineers to understand, they need to grasp the path and process of fault diagnosis solely through the output. To this end, our goal is to treat FL-Aler as a black box, ignoring the internal working principles of the model and only providing clear and easy-to-understand fault discrimination outputs. We use a simple model to fit the results of FL-Aler. By providing the path for determining the fault at each step, we can demonstrate the reliability of FL-Aler's results.

Overall, compared to other models, FL-Aler adopts a binary classification framework that reduces training parameters and model complexity. By learning from historical faults and dividing them into fault units, it can more effectively and quickly locate faults and provide reliable explanations. In terms of feature extraction, FL-Aler screens and aggregates spatial and temporal features, improving the quality of training data and thus enhancing the accuracy of fault localization.

3 FL-Aler Model

In this section, we first present an overview of the FL-Aler framework in Section 3.1. In the following, we delve into the details of the components included in FL-Aler, covering Sections 3.2 to 3.3.2.

3.1 Overview

In this section, we introduce the FL-Aler model. It includes a feature encoding component that extracts temporal and spatial features of faults, represented in Figure 2. The temporal features are derived from fault indicators at the time of occurrence, while spatial features capture correlations between fault units. These features are then processed through a multi-layer GAT to address dependencies caused by multi-hop fault propagation. In the fault localization component, a multi-head attention mechanism resolves complex dependencies among fault features. FL-Aler utilizes a binary classifier to assess each fault unit based on its unique and related features. Finally, to manage the imbalanced distribution of fault knowledge, it incorporates undersampling and a weighted KL divergence loss function.

3.2 Feature Encoding Component

When a fault occurs, the monitoring equipment issues an alert, and the recently captured fault indicators (usually set to 20 minutes) are output as input for the feature encoding component. The input information is first processed through temporal feature encoding to filter out a large number of feature indicators that are not useful for fault localization. Afterwards, spatial feature encoding is employed to capture spatial features. Finally, temporal features and spatial features are aggregated to obtain a complete fault feature for classification.

3.2.1 Temporal Feature Encoding. The extraction of temporal features involves three stages. In the first stage, a feature matrix is obtained by extracting fault feature indicators within a certain time window. Then, in the second stage, higher-level features that are more effective for fault localization are acquired. Finally, in the third stage, these features are further concatenated to obtain a fixed-length feature vector. This process fully extracts the most useful fault features for fault localization, resulting in a feature vector with a fixed length. It solves the problem that different fault units often contain multiple different fault features, and not all fault features are effective for fault localization.

First Step. Gated Recurrent Unit (GRU) recurrent neural network is used to extract the temporal features of fault units. GRU, a type of gated mechanism in recurrent neural networks, has a simple structure and fewer parameters. The input metrics of a fault unit are encoded into a numerical matrix with dimensions $W \times M$, where W is the length of the time window (usually set to 20 minutes) segmented based on fault time, and M is the number of metrics for the fault unit. When calculating the update gate and reset gate in the GRU, the update gate z_t concatenates x_t at timestep t with the information from timestep $t - 1$, performs a linear transformation (by multiplying with the weight matrix W_z), and then passes it through a sigmoid function to output a value within a fixed range of 0 to 1. The calculation process for the reset gate r_t is consistent with that of the update gate, and the calculation formulas are as follows:

$$z_t = \sigma(W_z \cdot [h_{t-1}, x_t]) \quad (1)$$

$$r_t = \sigma(W_r \cdot [h_{t-1}, x_t]). \quad (2)$$

This hidden state encompasses the temporal characteristics of the faulty component, reflecting the dynamic changes and correlations of the input features over time. Let's define z_t as the update gate, r_t as the reset gate, W_z and W_r as the corresponding weight matrices, σ as the Sigmoid function, and $[h_{t-1}, x_t]$ as the concatenation of the hidden state from the previous timestep and the current input vector.

The reset gate is multiplied by the hidden state output h_{t-1} from the previous timestep $t - 1$. The product determines how much information from the previous timestep can be utilized (the previously calculated reset gate is a vector composed of values ranging from 0 to 1, which measures the degree of gate opening). The result of this multiplication is concatenated with the new input x and undergoes a linear transformation, followed by a tanh activation function, to obtain the new $\tilde{h}_t$. The gate value $1 - z_t$ output by the update gate is multiplied by the hidden state output h_{t-1} at timestep $t - 1$ to determine the information from the previous timestep that is retained in the final memory. Subsequently, the gate value z_t output by the update gate is added to the result obtained in the third step (which represents the information from the current memory that is retained in the final memory) to produce the final hidden state output h_t . The calculation formulas are as follows:

$$\tilde{h}_t = \tanh(W \cdot [r_t \odot h_{t-1}, x_t]) \quad (3)$$

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t, \quad (4)$$

where $\odot$ represents element-wise multiplication, w is the weight matrices, and $\tilde{h}_t$ is the candidate hidden state of the fault unit at the current moment. Thus, the hidden state h_t obtained through the calculations of the update gate and reset gate contains the temporal features of the fault unit.

Second Step. FL-AIer applies a 1D convolutional neural network [38, 39] and **Gaussian Error Linear Unit (GELU)** activation function on the temporal feature matrix obtained in the first stage. The input feature matrix is M , the convolution kernel is K , a is cubic weight coefficient $a = 0.044715$, we set same as shayXu [40] and the bias term is b , which can be expressed as

$$W' = \text{ReLU}(M \times K + b) \quad (5)$$

$$Y = 0.5W' \left(1 + \tanh \left(\sqrt{\frac{2}{\pi}} \left(W' + a(W')^3 \right) \right) \right). \quad (6)$$

By applying 1D convolutional neural network, FL-AIer obtains multiple feature maps, capturing the correlations between different time points and metrics. The GELU activation function provides non-linearity by maintaining linearity in positive activation and suppressing negative activation, thus mitigating the problem of gradient vanishing.

Third Step. A fully connected layer is applied to learn the relationships between different feature maps obtained in the second stage, and outputs a unit-level feature vector of dimension Z . The weight matrix is W_q , and the bias term is b . The expression can be represented as

$$Z = \text{ReLU}(Y \cdot W_q + b). \quad (7)$$

After temporal feature encoding, we initially extract the fault indicator information of the faulty unit in the time dimension. This process enables us to capture the temporal patterns and trends of the fault indicators, which are crucial for accurate fault localization and classification. By analyzing the temporal features, we can gain insights into the behavior of the faulty unit over time, identifying patterns that may indicate the presence of a fault.

3.2.2 Spatial Feature Encoding. Spatial features are utilized to model the propagation of fault units. Given that a fault unit may trigger alterations in the fault characteristics of other units, and the relationship among fault units is diverse, FL-AIer incorporates spatial features into the feature matrix obtained in the previous step during the Spatial Feature Encoding phase. This results in the final feature matrix that FL-AIer utilizes for classification.

(1) *Calculate Fault Unit Relationship Weight.* It is clearly inappropriate to simply average the connected fault units as GraphSAGE does [41]. Therefore, FL-AIer uses the GAT to dynamically calculate the impact weights of different fault units on other fault units. The attention weights of each fault unit calculated by individual GAT can be represented as

$$\alpha_{ij} = \frac{\exp(\text{LeakyReLU}(a^T [Wh_i \| Wh_j]))}{\sum_{k \in N_i} \exp(\text{LeakyReLU}(a^T [Wh_i \| Wh_k]))}, \quad (8)$$

where Wh_i is a learnable weight matrix, used to linearly transform the features of neighboring nodes. a^T is a learnable parameter of the attention network, and $[\cdot \| \cdot]$ denotes the concatenation operation. The LeakyReLU function is employed as a non-linear function to enhance the model's expressive capability. N_i represents the set of neighboring fault units of fault unit i , and j is the fault unit node adjacent to i .

(2) *Modeling Multi-Hop Fault Unit Propagation.* In practice, the propagation of fault units may extend beyond one hop, as shown in Figure 1. Therefore, we use a multi-layer GAT, where the input of the previous layer is used as the output of the next layer to capture long-distance fault propagation. Translation of the academic paper excerpt: FL-AIer define Z_t as the feature vector

obtained through temporal feature encoding. Each GAT comprises H attention heads and there are L layers of GAT in total. The formula for spatial-temporal features from multi-layer GAT encoding is

$$f(Z_t) = [GAT_1(Z_t); \dots; GAT_L(Z_t)]. \quad (9)$$

(3) *Eliminating Network Training Over-Smoothing*. After multiple layers of **Graph Neural Networks (GNNs)**, each fault unit's fault index features and spatial structural features within a certain time period are aggregated into a fixed-length feature vector, solving the problem of different fault units often containing different fault metrics. Another major threat to the training of multi-layer GNNs is the problem of over-smoothing in deep networks. We introduce residual modules to mitigate this threat [52]. ResNet can reduce the number of model parameters and reduce complexity by using batch normalization layers instead of dropout, greatly reducing the amount of parameter calculations.

After processing through this component, we obtain the feature matrix of the faulty units, which provides input for the fault localization component to classify and train the model.

3.3 Fault Localization Component

In this section, we further introduce the fault localization component.

3.3.1 Fault Feature Dependency Elimination. To solve the complex dependency relationships caused by the propagation characteristics of fault features and to improve classification performance, our dependency handling component is divided into three stages:

(1) *Positional Encoding Module*. We take the spatial-temporal feature vectors obtained from the feature encoding component and perform position encoding to introduce positional information into the feature vectors:

$$PE_{(pos, 2i)} = \sin\left(pos/10000^{\frac{2i}{d}}\right), \quad (10)$$

$$PE_{(pos, 2i+1)} = \cos\left(pos/10000^{\frac{2i}{d}}\right). \quad (11)$$

Compared to relative position coding, direct position coding is based on fixed reference point coding, which is simple to implement and has stable results. This is beneficial for the model to learn and process the sequential relationships in the input sequence.

(2) *Multi-Head Self-Attention Module*. In this stage, FL-AIer calculates the query matrix Q , key matrix K , and value matrix V based on the input feature matrix X . The expressions are

$$Q = XW^Q, \quad (12)$$

$$K = XW^K, \quad (13)$$

$$V = XW^V. \quad (14)$$

The generated normalized attention weight matrix undergoes a softmax nonlinear transformation to obtain the output matrix Z of the self-attention mechanism. The dimensions of matrices Q and K are used as scaling factors to ensure that the inner product QK^T does not become excessively large and that the softmax function operates smoothly. Its structure is shown in Figure 3. The use of multiple attention heads enables the current model to focus on features from different positions. Compared to a two-layer dense neural network, it can better focus on the interdependencies between each metric in the aggregated features. In summary, the output suspicious score of a fault unit can be represented as

$$score = Attention(Q, K, V) = softmax\left(\frac{QK^T}{\sqrt{d_K}}\right)^V, \quad (15)$$

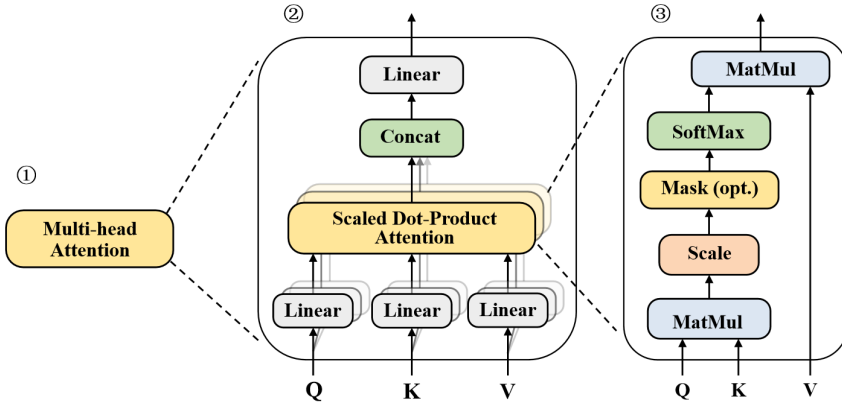


Fig. 3. The detail of the multi-head self-attention model.

where d_K is the number of columns in matrices Q and K , which represent the dimension of the vector.

(3) *Add and Norm Module*. Through the Add and Norm layer, addition and layer normalization are applied to the input, enhancing the model's training stability and accelerating the training process. Specifically, the addition operation is divided into two steps: In the Addition Operation, the model adds the input to the residual connection. A residual connection directly passes the input to the layer's output without extra transformations. This mitigates the vanishing gradient problem, facilitating the learning of residual information during backpropagation.

After the addition operation, the results undergo layer normalization, which standardizes each feature of every sample so that it has a mean of 0 and a standard deviation of 1. This helps to stabilize the training process and mitigates the effects of internal covariate shift.

Through this component, we effectively address Challenge 1 and eliminate the complex dependency issues of fault features.

3.3.2 Fault Knowledge Balance. In this section, we introduced the Fault Knowledge Balancing module. In real-world scenarios, models often bias towards frequent fault types due to the uneven occurrence of different faults and the dominance of non-fault units in training data. To combat this, we designed a Fault Knowledge Balance Component that employs oversampling to equalize fault representation and a weighted KL divergence loss function to prioritize rare or significant faults. This approach ensures balanced learning and improves the model's ability to diagnose diverse fault types by calculating specific scores for each fault unit in the output layer, enhancing predictive accuracy for fault detection and diagnosis.

In practice, fault frequencies vary across categories. To ensure FL-AIer doesn't neglect minority categories, we over-sample historical faults in these during training. With C fault categories and N_i faults in the i th category, each fault is sampled with a probability of $\frac{1}{C \cdot N_i}$. This equalizes sampling probabilities across categories (i.e., $1/C$), fostering balanced training and avoiding oversight of less common faults.

Furthermore, FL-AIer employs a weighted KL divergence loss function to tackle the skewed distribution of faults. It aims for a fault unit v in fault T to have a suspicious score $S_T(v) \in [0, 1]$ closely aligning with the true label $r_T(v) \in [0, 1]$. The difference between $S_T(v)$ and $r_T(v)$ is measured using the KL loss, enhancing fault localization effectiveness. FL-AIer calculates loss using KL loss by

$$D_{KL}(S_T(v) || r_T(v)) = \sum_{i=1}^N [S_i(v) \log S_i(v) - S_i(v) \log r_i(v)] . \quad (16)$$

In most cases, the majority of fault units remain fault-free, which makes their losses more significant compared to units with faults. This can lead the model to treat all fault units equally when scoring them. To tackle this issue, FL-AIer assigns different weights to faulty and normal fault units. Specifically, a faulty unit is assigned a weight equal to the total number of normal fault units, while normal units have a weight of 1. This results in the following representation of the overall loss function:

$$w_v^{(T)} = R_T(v) \cdot |V| + (1 - R_T(v)), \quad (17)$$

$$loss = \frac{1}{N} \sum_T \left[\frac{\sum_{v \in V} w_v^{(T)} D_{KL}}{\sum_{v \in V} w_v^{(T)}} \right], \quad (18)$$

where V is the set of failure units, $T \in \{T_1, \dots, T_H\}$ and $\{T_1, \dots, T_H\}$ is the set of training failures, and H is the number of faults learned.

After this section, we have successfully resolved the issue of unbalanced fault knowledge and assigned scores to suspicious faulty units. This score serves as a quantitative measure of the likelihood of a unit being faulty, enabling us to prioritize and address potential issues more effectively. By addressing the imbalance in fault knowledge, we can ensure that our model is not biased towards certain types of faults, leading to more accurate and reliable fault detection and diagnosis.

4 Interpretation Methods

Although many models already have interpretation methods, they aim to explain deep learning models by understanding their internal mechanisms [42]. However, in industrial practice, engineers are mainly concerned about the relationship between raw data and root causes. Therefore, we propose a new interpretation method. Our approach aims to utilize a simpler yet highly interpretable model to effectively approximate the performance of the complex FL-AIer model. In this process, we select **Decision Trees (DTs)** as the surrogate model, with the input being a feature matrix carefully processed by the feature encoding component and the target variable being the suspicious score ultimately output by the FL-AIer model.

Prior to inputting the feature matrix into the DTs, we face a crucial issue: the dimensionality setting of fault feature indicators for fault units. Each fault unit contains multiple fault feature indicators, but not all of them play a decisive role in fault localization. Complicating the matter further, different fault units often contain different sets of indicators. To address this issue, we design a decoder within the temporal feature encoding module of the feature encoding component.

The model of this decoder is illustrated in Figure 4. The decoder comprises a one-dimensional transpose convolution (1D DeConv) layer, which can be treated as a transpose operation of Conv1D, and a fully connected layer. It is trained by minimizing the mean square error between the original and reconstructed metrics. If a time-series feature differs greatly between the original and reconstructed metrics, it indicates that the feature is not retained by the FL-AIer model and should be removed.

Utilizing these screened and encoded features, we train DT models to accurately classify units as “faulty” or “normal.” When the suspicious score of a failure unit is greater than 0.9, we define it as “faulty.” When the suspicious score is less than 0.1, we define it as “normal.” Otherwise, we do not handle it. By deeply analyzing the decision paths leading to these classifications, we gain a deeper understanding of the reasoning logic and working principles of FL-AIer.

Although DTs may fall slightly behind FL-AIer in terms of performance and may occasionally make errors, they provide intuitive and easily understandable insights for fault diagnosis. This enhances engineers’ trust in FL-AIer, enabling them to rely on the model more confidently for fault localization and analysis.

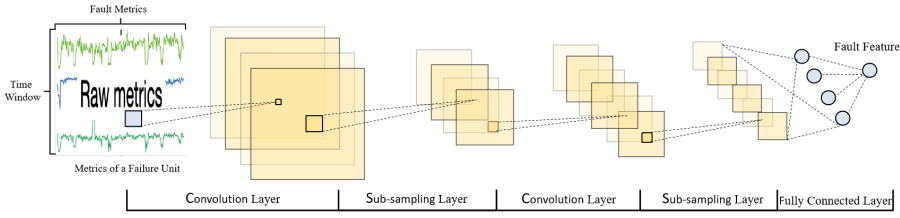


Fig. 4. The framework of the decoder model.

Additionally, we further explain how FL-AIer performs fault diagnosis by analyzing which historical faults it learns from to identify the current faulty unit. This is achieved by comparing the incoming fault with each historical fault using the aggregated features extracted by FL-AIer. Notably, we compare fault categories rather than individual fault units, as similar faults may occur at different locations within the system.

5 Experiments

5.1 Research Questions (RQs)

Our evaluation addresses the following RQs:

- RQ1: How does FL-AIer perform in fault localization?
- RQ2: Is FL-AIer more suitable for industrial fault localization?
- RQ3: How does FL-AIer perform with limited data?
- RQ4: What is the contribution of each component of FL-AIer?
- RQ5: How does FL-AIer perform under different parameter settings?
- RQ6: How is the generalization performance of FL-AIer?

RQ1 is used to evaluate the effectiveness of FL-AIer in classifying benchmark datasets. RQ2 aims to demonstrate the superiority of the FL-AIer model over the traditional Random Forest classifier in the practice of industrial fault localization. RQ3 is aimed at assessing the generalization ability of FL-AIer. RQ4 aims to explore the contribution of each component in FL-AIer to its performance. RQ5 is used to evaluate the impact of different parameters on FL-AIer. RQ6 is designed cross-experimental validations using four datasets to assess the generalization ability of the model.

5.2 Experimental Setup

5.2.1 Datasets. We use four datasets [13] from sys_A (a major ISP's microservice system), sys_B (a commercial bank's service-oriented system), sys_C (a commercial bank's Oracle database), and sys_D (an open source microservice benchmark system [17]). As shown in Table 1, the four datasets contain a total of 601 failures. Specifically, 16 failures have multiple root causes, resulting in multiple faulty units. There are 1,376 faulty units in total. We define the same type of failures that occur in different locations as one failure class, and there are 102 failure classes in total. All faulty units have a total of 11,447 metrics. sys_A , sys_B , sys_C are from production systems, and sys_D is an open source benchmark. We randomly divide each dataset into training set (40%), validation set (20%), and test set (40%) for each experiment.

Datasets sys_A , sys_B , and sys_D have real scenarios determined by fault injection location and type. sys_A and sys_B had 10 types of faults injected, including CPU exhaustion, packet loss/delay, database session limitations, low free memory, and disk I/O exhaustion. sys_D used ChaosMesh to inject faults, covering CPU/memory stress, pod/node failures, and packet corruption/loss/delay. sys_C contains 99 real faults, each fault class having only one fault unit, annotated by experienced engineers.

Table 1. Dataset Details

Datasets	Failures	Metrics	Failure Units	Failure Classes	System	Failure Source
<i>sys_A</i>	188	710	102	18	A production microservice system of a major ISP	Injected by the engineers
<i>sys_B</i>	158	2,419	189	20	A production service-oriented system of a commercial bank	Injected by the engineers
<i>sys_C</i>	99	2,594	41	41	A production Oracle database system of a commercial bank	Real-world failures
<i>sys_D</i>	156	5,724	1,044	23	An open source microservice benchmark system, Train-Ticket	Injected by Li et al. [13]

5.2.2 Baselines. To validate the effectiveness of FL-AIer, we compare it with the state-of-the-art fault localization models: DéjàVu [13] utilizes historical faults and fault dependency graphs to score and recommend fault types and locations to engineers; Brandón et al. [21] represent faults as graphs and find similar diagnosed faults through graph similarity; iSQUAD [24] clusters historical faults, labels root causes, and assigns incoming faults to clusters; Traditional Machine Learning Models (DT, Gradient Boosting, Random Forest, Support Vector Machine) use time series features to detect faults; Random Walk [28, 43] is used in unsupervised fault localization. Two RW variants are compared: RandomWalk@Metric and RandomWalk@FI.

5.2.3 Evaluation Metrics. We evaluate effectiveness using top- K accuracy ($A@K$) and MAR, consistent with related work. $A@K$ indicates the likelihood of identifying the root cause by inspecting the top K suspected fault units, with higher values being better. MAR represents the average rank of all ground truths for each failed fault, signifying how many suspected fault units, on average, need examination to diagnose a fault, with lower values being better.

5.3 Training Details

The experimental environment is a desktop computer equipped with an NVIDIA GeForce RTX 3090 GPU and an Intel Core i9-10940X CPU, running on the Ubuntu 20.04 operating system. We implemented FL-AIer and all state-of-the-art methods based on PyTorch and DGL [44]. We used RMSprop to train the FL-AIer model, with the initial learning rate set to 0.01 and weight decay set to 0.01. To prevent gradient explosion, we clipped the gradients of all parameters to 1. We trained each model for a maximum of 3,000 epochs and employed early stopping, with a batch size set to 16. Our implementations, including state-of-the-art methods, can be found in our replication package [18].

6 Experimental Results

6.1 How Does FL-AIer Perform in Fault Localization?

Motivation. The objective of this experiment is to verify the effectiveness of FL-AIer in fault localization and compare its performance with that of baseline methods.

Methodology. To verify the effectiveness of FL-AIer in fault localization, we replicate the experimental setup and steps of the state-of-the-art methods and compared them with these state-of-the-art methods; we not only compared it with state-of-the-art methods and other existing models that use historical faults for localization, but also constructed the best localization results obtained by traditional classifiers after a series of parameter tuning to compare with FL-AIer. Due to random training, we repeated each experiment 10 times and present the average results.

Results. Table 2 presents the results of fault localization. The experimental results demonstrate that FL-AIer outperforms state-of-the-art methods in terms of code summarization performance, as evidenced by metrics such as $A@1$, $A@2$, $A@3$, $A@5$, MAR, and so on. For instance, compared to state-of-the-art methods, on *sys_A*, FL-AIer showed improvements in $A@1$ by 5.82%, 390.22%, 254.27%, 53.52%, 28.37%, 11.31%, 158.94%, 1,520.44%, and 394.67%. On *sys_C*, the improvements in

Table 2. Comparison Results of FL-Aler with the State-of-the-Art Methods

Datasets	Method	A@1	A@2	A@3	A@5	MAR	Datasets	Method	A@1	A@2	A@3	A@5	MAR
sys _A	Brandón	16.66	25.64	35.89	47.44	24.31	sys _C	Brandón	34.21	52.63	60.53	68.42	26.34
	iSQUAD	23.07	30.77	39.74	46.16	12.91		iSQUAD	31.58	57.89	63.16	76.32	3.50
	DT	53.20	63.46	65.77	66.03	18.05		DT	38.95	44.74	45.27	58.42	11.21
	Gradient Boosting	63.62	72.49	76.18	81.35	3.82		Gradient Boosting	44.74	67.37	73.68	77.11	3.09
	Random Forest	73.37	89.15	91.94	95.71	1.88		Random Forest	61.05	75.26	82.11	87.10	2.26
	SVM	31.54	39.05	44.53	51.34	7.66		SVM	26.84	46.32	60.26	73.69	4.09
	RandomWalk@Metric	5.04	8.89	12.74	26.73	11.26		RandomWalk@Metric	10.53	18.42	34.21	47.37	8.25
	RandomWalk@FI	16.51	19.04	20.32	20.32	33.64		RandomWalk@FI	23.68	36.84	42.11	52.63	7.08
	harmonic [45]	28.94	45.74	52.87	62.51	15.36		harmonic	30.92	36.99	44.20	51.82	10.36
	jaccard [46]	35.13	44.24	49.89	60.53	17.85		jaccard	40.11	41.15	49.59	58.16	16.53
	tarantula [47]	27.58	40.41	53.53	68.63	30.18		tarantula	19.54	26.28	40.12	49.65	15.06
	naish2 [48]	16.58	23.14	39.25	56.01	50.36		naish2	16.25	29.43	31.15	48.15	48.26
	dstar [49]	43.18	50.27	60.25	68.98	24.19		dstar	30.19	40.18	52.19	58.29	19.25
	DéjàVu	77.18	90.38	93.40	96.28	1.66		DéjàVu	61.84	82.63	90.79	96.32	1.70
	GDSV [50]	52.14	65.89	68.76	73.72	4.03		GDSV	61.82	69.56	71.34	76.39	3.63
	devign [51]	60.18	65.39	70.12	74.22	4.08		devign	61.29	67.53	72.24	75.54	5.38
	FL-Aler	81.67	93.76	95.88	98.25	1.42		FL-Aler	64.42	87.11	92.90	95.00	2.70
sys _B	Brandón	15.15	19.70	24.24	30.30	47.92	sys _D	Brandón	8.70	8.70	8.70	13.04	303.39
	iSQUAD	7.58	10.61	15.15	24.24	30.75		iSQUAD	21.74	30.43	30.43	31.88	259.07
	DT	55.91	60.30	63.49	65.00	30.70		DT	64.37	72.19	73.13	73.60	128.98
	Gradient Boosting	72.00	75.23	76.77	80.00	8.77		Gradient Boosting	50.72	59.42	59.42	63.77	33.79
	Random Forest	84.46	85.84	87.84	88.92	8.75		Random Forest	85.66	86.67	89.13	91.88	12.53
	SVM	0.15	1.23	1.54	12.15	23.30		SVM	1.45	7.25	10.14	15.94	92.35
	RandomWalk@Metric	9.38	20.31	28.12	31.25	15.12		RandomWalk@Metric	2.90	5.80	8.70	13.04	104.89
	RandomWalk@FI	21.88	21.88	28.12	40.62	30.66		RandomWalk@FI	0.00	0.00	0.00	1.45	443.79
	harmonic	30.13	46.84	50.50	65.53	15.85		harmonic	30.62	37.38	44.52	51.35	10.72
	jaccard	25.19	37.11	49.26	58.16	26.31		jaccard	25.16	33.56	39.74	50.49	36.74
	tarantula	26.19	29.98	36.18	40.53	26.37		tarantula	26.10	29.85	35.89	40.28	26.33
	naish2	40.26	46.84	50.50	65.53	15.85		naish2	26.64	28.86	35.04	40.05	26.69
	dstar	39.76	44.69	49.42	67.23	15.47		dstar	41.88	46.85	50.28	65.61	15.17
	DéjàVu	66.21	71.21	75.61	79.24	5.03		DéjàVu	75.62	84.69	90.42	94.27	2.63
	GDSV	58.14	63.89	70.76	76.20	5.26		GDSV	61.82	69.56	71.34	79.39	3.58
	devign	60.15	66.29	70.19	74.26	6.18		devign	59.31	62.49	70.16	74.82	5.03
	FL-Aler	73.34	77.19	81.72	85.31	4.2		FL-Aler	87.39	92.48	94.78	96.77	3.20

Boldface indicates the optimal experimental results obtained by various methods under the corresponding indicators.

ACM Transactions on Software Engineering and Methodology, Vol. 34, No. 6, Article 165. Publication date: July 2025.

A@1 are 4.17%, 88.31%, 103.99%, 65.39%, 43.99%, 5.52%, 140.01%, 511.78%, and 172.04%. On sys_D , the increases are 15.56%, 905.63%, 301.98%, 35.76%, 72.30%, 2.02%, 5,926.90%, and 2,913.45%. FL-AIer also significantly surpassed state-of-the-art methods in other measurement metrics. Further calculations show that FL-AIer's average A@1 is 76.45%, an improvement of 9.24% over the state-of-the-art methods. Compared to Brandón and iSQUAD, FL-AIer performs better because it learns from historical failures and adopts a supervised training approach, which makes it less susceptible to unrelated interference factors from other metrics and has strong resistance to noise and fluctuations. Traditional machine learning models perform poorly due to the existence of a large number of useless and noisy time-series features. The two random walk-based methods are difficult to train when the number of failure metrics is large, and they are also unsupervised and susceptible to noise.

Compared to GDSV, the FL-AIer method achieved improvements in the A@1 metric by 56.63%, 26.14%, 4.21%, and 41.36%, respectively, across the four datasets. When compared to devign, FL-AIer demonstrated improvements in the A@1 metric by 35.71%, 21.93%, 5.11%, and 47.34%, respectively, on the same datasets. The reason for the underperformance of GNNs lies in the fact that each faulty unit has multiple fault indicators, which makes the model susceptible to noise and indicator fluctuations. Additionally, GNNs can only capture relationships between adjacent nodes and are unable to effectively handle the complex dependencies arising from fault propagation.

Compared to harmonic, jaccard, tarantula, naish2, and dstar methods, the FL-AIer method improved by 182.20%, 143.41%, 108.34%, and 185.40%, respectively, on the A@1 indicator across the four datasets compared to the harmonic method (note: there seems to be a repetition in the comparison with harmonic, which should be corrected if it's a typo), 132.48%, 191.15%, 60.61%, and 247.34% compared to the jaccard method, 196.12%, 180.03%, 229.68%, and 234.83% compared to the tarantula method, 392.58%, 82.16%, 297.66%, and 228.04% compared to the naish2 method, and 89.14%, 84.46%, 113.38%, and 110.51% compared to the dstar method. The traditional methods such as harmonic, jaccard, tarantula, naish2, and dstar performed poorly because they adopt unsupervised approaches that are susceptible to noise interference. Additionally, they struggle to handle high-dimensional time-series dataset features. Furthermore, the ochiai method has difficulties in dealing with complex dependencies between fault features.

Answering RQ1: Overall, FL-AIer performs well across four datasets and five evaluation metrics. In different scenarios, compared to unsupervised methods, FL-AIer has stronger noise and interference resistance, eliminates the impact of irrelevant abnormal changes in other indicators, and improves upon the issues of insufficient or inaccurate fault feature extraction in traditional machine learning methods.

6.2 Is FL-AIer More Suitable for Industrial Fault Localization?

Motivation. In Section 6.1, although FL-AIer is significantly better than other models in most measurement indicators, it is slightly worse than the random forest method on sys_B . Therefore, it is necessary to further investigate whether FL-AIer is more suitable for industrial fault localization, compared to random forest and current state of the art methods.

Methodology. We compared the time consumption of locating the same faulty unit on four datasets.

Results. As shown in Table 2, aside from the Random Forest method, FL-AIer shows improvements over other methods on sys_B in A@1, with increases of 10.77%, 384.09%, 837.58%, 31.18%, 1.86%, 48,793.33%, 292.32%, and 235.19%. Additionally, FL-AIer achieves the best performance in MAR, scoring 4.2, which is a 16.5% improvement over Random Forest. This highlights that Random Forest performs very poorly in certain extreme scenarios and tends to favor the training dataset. In

Table 3. Time Efficiency Comparison

Positioning Time (Seconds)	<i>sys_A</i>	<i>sys_B</i>	<i>sys_C</i>	<i>sys_D</i>
Random Forest	0.28	0.79	0.46	0.78
DéjàVu	0.09	0.07	0.08	0.05
FL-AIer	0.07	0.06	0.07	0.03

Boldface indicates the optimal experimental results obtained by various methods under the corresponding indicators.

contrast, FL-AIer demonstrates strong overall performance. Further, the development of FL-AIer is informed by the inefficient and time-consuming nature of industrial fault localization. We analyzed the time taken by both methods for localizing each fault unit. As shown in Table 3, on four datasets, the times for FL-AIer are 0.07 s, 0.06 s, 0.07 s, and 0.03 s, saving 75%, 92.4%, 84.88%, and 96.15% of the time compared to the Random Forest method, respectively. Compared to the Random Forest method, FL-AIer performs better because Random Forest requires time-consuming time series feature extraction, which makes it impractical for industrial fault localization. In contrast, FL-AIer learns directly from historical fault units, making it efficient and more scalable.

Methodology. We will compare our interpretation method with two widely used methods **Logistic Regression (LR)** and **LIME**. We provide a DT for our OS network fault interpretation method. For LR, we train and interpret LR models with DTs that have identical features and objectives, and use feature coefficients as feature importance. LIME explains individual predictions by learning local linear approximations of the model.

Results. Compared to the DT model of FL-AIer, LR and LIME cannot provide explanatory rules, only importance, which is not explanatory or interpretable. In addition, LIME provides very different measurement importance for similar faults, which can be confusing. We present the scoring of the top five features with the greatest impact on the fault unit OS network for dataset *sys_A* using LR and LIME. It can be shown in Figure 5, the top five features they obtained are not exactly the same, and only the importance scores of the features are given, which cannot provide engineers with a reasonable explanation for such results. The DT model of FL-AIer is shown in Figure 6. When fault 6 occurs, we will obtain a complete path of the fault generation, which is normal through the transmission queue 1, 2, but a fault occurs at step 5 *ss_{total}*. Overall, FL-AIer displays reasonable rules for discovering network related issues.

Answering RQ2: In industrial practice, in addition to accurately locating the type of fault, we usually hope to quickly locate the fault (as detailed in Section 2.1) and provide a reasonable explanation (as described in Sections 2.1 and 4), so that engineers can trust the suspicious faults provided by our model. Therefore, based on the above experiments, it can be concluded that FL-AIer is more suitable for locating real faults in industrial scenarios, suitable for various system environments, less affected by extreme situations, can significantly reduce fault localization time, and has better interpretability, which can provide engineers with more reliable suspected faults.

6.3 How Does FL-AIer Perform with Limited Data?

Motivation. Collecting fault datasets in industrial practice is difficult because some faults may occur at a lower frequency and the model may not receive sufficient training. To demonstrate

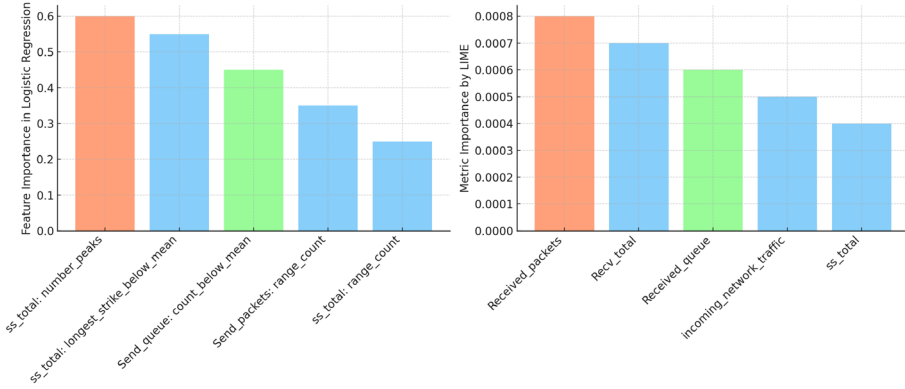


Fig. 5. The scores of the top five indicators for LIME and LR. LR, Logistic regression.

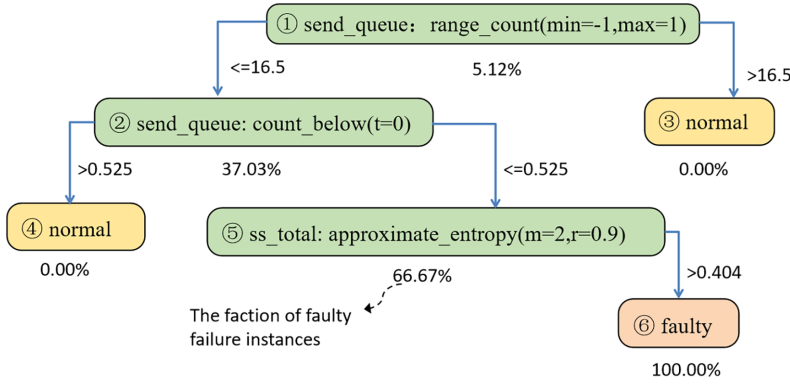


Fig. 6. A DT trained for the failure class OS network of sys_A .

the generalization ability of FL-AIer, we conduct experiments on various datasets to study the performance of FL-AIer with insufficient training data.

Methodology. We compared the performance of FL-AIer's various metrics across a range of 10% to 100% of the original training data.

Results. The experimental results are shown in Table 4. In dataset sys_A , when using 70% of the faulty dataset, the A@k and MAR values reached 81.25%, 93.75%, 93.75%, 96.88%, and 8.36%, respectively. Compared with the best results, the A@k and MAR values decreased by only 2.36%, 0.06%, 2.22%, and 1.39%, respectively. When using 80% of the fault dataset, the best result of 93.81% can be achieved on A@2. When using 80% of historical fault data in dataset sys_B , indicators such as A@2 and MAR reach their maximum values (i.e., 78.34%, 3.86%), which are 1.49% and 4.22% higher than the complete dataset with the same indicators, respectively. When using 90% of the dataset in dataset sys_C , the best values can be achieved on A@2, A@3, and A@5, which are 89.67%, 92.9%, and 95.2%. When using 80% of the dataset in dataset sys_D , the best value of 89.53% and 93.18% can be achieved on A@1 and A@2. FL-AIer can achieve good performance with only 80% of the training data in each dataset, because FL-AIer has excellent generalization performance. Based on the above data analysis, it can be seen that even when the dataset is insufficient, FL-AIer can still provide reliable fault recommendations, because FL-AIer fully captures the most useful fault features and eliminates the dependency relationship between fault feature indicators. Although MAR values

Table 4. Performance Metrics of the FL-Aler Model with Varying Proportions of Historical Faults Retained

Datasets	Dataset Size	A@1	A@2	A@3	A@5	MAR	Datasets	Method	A@1	A@2	A@3	A@5	MAR
<i>sys_A</i>	10%	7.58	10.61	15.15	24.24	53.14	<i>sys_C</i>	10%	3.78	7.29	7.29	10.53	91.11
	20%	13.44	36.84	50.86	63.75	42.58		20%	9.52	9.52	9.52	16.38	104.66
	30%	45.58	53.52	71.30	84.21	30.20		30%	19.38	19.38	32.68	65	50.52
	40%	49.35	68.11	85.63	89.56	31.59		40%	26.51	30.4	50.21	60.54	70.38
	50%	60.56	71.05	84.21	89.48	23.42		50%	38.74	55.77	63.49	74.39	56.94
	60%	70.41	89.58	90.40	94.37	19.52		60%	45.68	60.3	78.51	85.25	41.37
	70%	81.25	93.75	93.75	96.88	8.36		70%	57.25	70.33	90.60	90.6	20.17
	80%	80.65	93.81	93.26	95.77	4.38		80%	55.91	80.19	89.37	92.11	5.18
	90%	83.21	92.06	94.32	96.39	1.44		90%	60.38	89.67	92.90	95.2	3.19
	100%	81.67	93.76	95.88	98.25	1.42		100%	64.42	87.11	92.90	95.00	2.70
<i>sys_B</i>	10%	7.19	8.45	10.11	10.11	63.66	<i>sys_D</i>	10%	8.50	10.61	15.15	24.24	69.72
	20%	15.15	19.7	24.24	30.3	47.92		20%	10.15	20.57	26.54	40.39	66.48
	30%	31.56	35.89	47.44	57.11	45.46		30%	26.49	35.16	45.59	54.228	42.17
	40%	47.73	53.20	63.46	65.77	30.70		40%	34.21	52.63	60.53	68.42	26.34
	50%	55.03	60.30	63.49	65.00	16.40		50%	55.71	59.97	61.72	66.58	19.69
	60%	63.21	68.58	72.36	76.06	11.21		60%	60.5	73.25	79.35	79.35	15.32
	70%	66.41	72.03	77.50	81.72	5.59		70%	80.6	86.26	86.26	86.46	9.66
	80%	71.25	78.34	79.38	82.03	3.86		80%	89.53	93.18	93.18	93.18	3.8
	90%	72.46	78.13	81.88	81.33	4.11		90%	87.98	92.48	94.79	95.33	4.16
	100%	73.34	77.19	81.72	85.31	4.03		100%	87.39	92.48	94.78	96.77	3.20

Boldface indicates the optimal experimental results obtained by various methods under the corresponding indicators.

typically increase with increasing dataset size in various datasets, and the best MAR values can only be obtained when using a complete fault dataset, the possible reason is that insufficient datasets reduce the stability of the model and may result in some extremely poor results. However, the MAR values on all four datasets can reach 5.18 or above, when the dataset reaches 80%.

Answering RQ3: Although not all metrics reach their optimum, it demonstrates that FL-Aler has good generalization ability. Even the dataset is not very extensive, good performance can be achieved with less training data.

6.4 What Is the Contribution of Each Component of FL-Aler?

Motivation. To verify the contribution of each component of FL-Aler, we design this experiment by removing each main module in FL-Aler to study their contributions.

Methodology. In this experiment, our complete model serves as the state-of-the-art method, while FL-Aler_{nonKL}, FL-Aler_{nonAtt}, and FL-Aler_{nonRes}, respectively, remove the weighted KL divergence loss function from the fault localization prediction component, remove the attention module in the dependency handling component, and remove the residual module.

Results. As shown in Table 5, experimental results by four datasets are as follows.

Compared to FL-Aler_{nonKL}, FL-Aler improves by 1.14%, 5.24%, 28.20%, and 9.67%, respectively, in A@1 on FL-Aler_{nonAtt}. Although the performance on *sys_A* is average, possibly because the model used downsampling to a certain extent to compensate for fault knowledge imbalance.

Compared to FL-Aler_{nonAtt}, FL-Aler improved by 12.57%, 3.84%, 24.87%, and 19.50%, respectively, in A@1 on various datasets. Although there is a decline of 2.03% and 0.63% in A@2 and A@3 on *sys_B*, it performs well on other metrics, possibly due to the presence of multiple root causes of failures in *sys_B*. Overall, the attention module effectively handles complex fault unit dependencies.

Compared to FL-Aler_{nonRes}, FL-Aler's MAR on each dataset increased by 12.88%, 55.03%, 17.22%, and 29.48%, respectively, and other evaluation metrics also improved. Since MAR indicates the

Table 5. Ablation Experiments of FL-AIer

Datasets	Method	A@1	A@2	A@3	A@5	MAR	Datasets	Method	A@1	A@2	A@3	A@5	MAR
sys_A	FL-AIer _{nonKL}	72.55	88.18	93.02	96.77	1.90	sys_C	FL-AIer _{nonKL}	50.79	78.16	90.53	96.84	2.10
	FL-AIer _{nonAtt}	80.75	92.36	95.02	96.57	1.96		FL-AIer _{nonAtt}	49.47	73.16	84.21	93.95	2.79
	FL-AIer _{nonRes}	81.97	91.58	93.85	96.34	1.63		FL-AIer _{nonRes}	62.37	87.1	92.37	94.74	3.31
	FL-AIer	81.67	93.76	95.88	98.25	1.42		FL-AIer	64.42	87.11	92.90	95.00	2.74
sys_B	FL-AIer _{nonKL}	70.63	79.22	82.35	84.85	4.71	sys_D	FL-AIer _{nonKL}	73.13	83.31	90.20	94.27	3.68
	FL-AIer _{nonAtt}	69.69	75.16	77.66	82.19	9.90		FL-AIer _{nonAtt}	79.68	87.34	92.98	95.42	4.26
	FL-AIer _{nonRes}	67.03	75.31	78.28	82.28	9.34		FL-AIer _{nonRes}	79.56	89.25	92.81	92.50	4.58
	FL-AIer	73.34	77.19	81.72	85.31	4.20		FL-AIer	87.39	92.48	94.78	96.77	3.23

Boldface indicates the optimal experimental results obtained by various methods under the corresponding indicators.

average number of recommendations needed to diagnose a failure, it is susceptible to the influence of extremely bad cases. In the case of FL-AIer_{nonRes}, a gradient explosion issue occurs, causing significant fluctuations in the training process.

Answering RQ4: In summary, the dependency handling component plays an important role in alleviating the complex dependency relationships of fault units, and the fault localization prediction component can better focus on addressing the problem of imbalanced distribution of fault knowledge in units with fewer occurrences. Each module in the FL-AIer model plays a significant role in the performance of the model, effectively improving its performance.

6.5 How Does FL-AIer Perform under Different Parameter Settings?

Motivation. Our model is influenced by a variety of different hyperparameters, and in order to find the optimal parameter settings, we verify the performance of different values of feature dimension (Z), GAT heads (H_1) and layers (L), attention heads (H_2), and the learning rate.

Methodology. We set Z to 3, H_1 to 4, L to 8, H_2 to 6, and the learning rate to 0.01 as the default values for our model. When analyzing the settings of each parameter, we used the control variable method. When examining the number of GAT heads, we set the number of heads H_1 to 1, 2, 4, and 8, respectively. When studying the impact of the number of attention heads, we adjusted the number of attention heads H_2 to 1, 2, 4, 6, and 8, respectively. When exploring the feature dimension, we set the dimension Z to 3, 6, 9, and 12, respectively. When considering the number of GAT layers, we set the number of layers L to 1, 2, 4, 8, and 16, respectively. For other variables, we adopted the default values. Then we recorded the values of A@1 under various circumstances on four datasets. Since the initial learning rate did not change significantly during the process of modification, the experimental results were not presented in the table.

Results. The variation of A@1 with different values of feature dimension (Z), GAT heads (H_1) and layers (L), attention heads (H_2) is illustrated in Figure 7.

When H_1 is set to 4, two datasets (sys_A and sys_D) achieve optimal values for A@1, with the best values being 81.67% and 87.39%, respectively. Although on dataset B, $H_1 = 4$ is 0.96% lower than

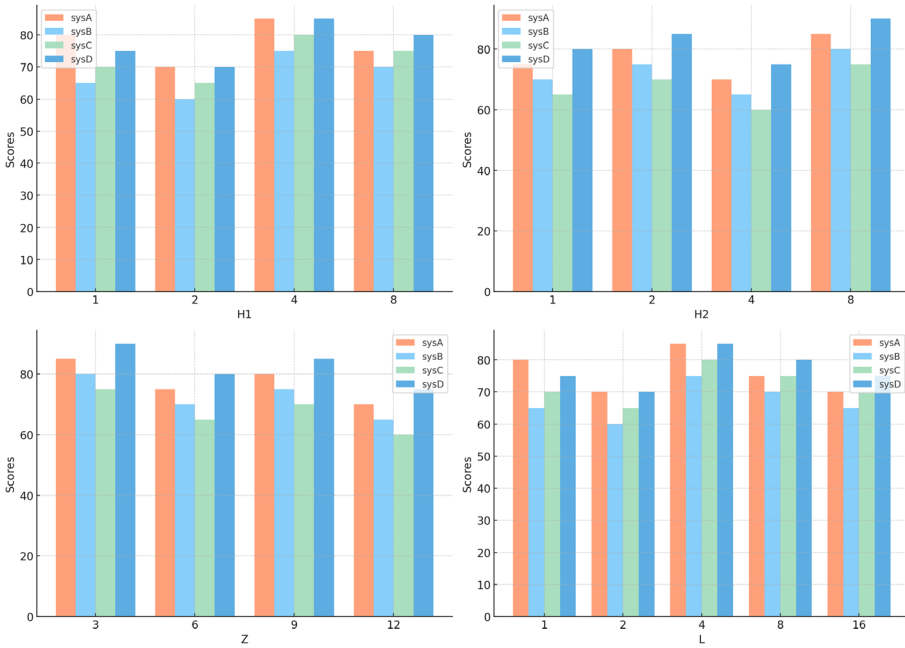


Fig. 7. Ablation experiments of FL-Aler.

the optimal value of $H_1 = 8$, and on dataset sys_C , $H_1 = 4$ is 0.89% lower than the optimal value of $H_1 = 2$.

Setting H_2 to 6 yields the best A@1 on datasets sys_A , sys_B , sys_C , and sys_D , outperforming other settings by 1.54%, 9.73%; 2.46%, 11.31%; 4.52%, 9.25%; and 2.54%, 7.25%, respectively. The optimal training performance is achieved with H_2 at 6.

When Z is set to 3, two datasets (sys_C and sys_D) achieve optimal values for A@1, with the best values being 64.42% and 87.39%, respectively. Although on dataset sys_A , $Z = 3$ is 1.76% lower than the optimal value of $Z = 9$, and on dataset sys_B , $Z = 3$ is 1.24% lower than the optimal value of $Z = 6$.

When L is set to 8, two datasets (sys_C and sys_D) achieve optimal values for A@1, with the best values being 64.42% and 87.39%, respectively. Although on dataset sys_A , $L = 8$ is 0.6% lower than the optimal value of $L = 1$, and on dataset sys_B , $L = 8$ is 0.2% lower than the optimal value of $L = 2$.

Answering RQ5: According to the experimental results, we optimized our model by setting parameters Z , L , H_1 , and H_2 to 3, 4, 8, and 6, and a learning rate of 0.01. These adjustments enhance training effectiveness, balancing efficiency and accuracy.

6.6 How Is the Generalization Performance of FL-Aler?

Motivation. Consistent with the verification schemes of existing works, we adopted the same measurement metrics as the current state-of-the-art methods. Moreover, we repeated each experiment 10 times and took the average value as the final result to further demonstrate the generalization ability of our model. Specifically, we designed cross-experiments using these four datasets. To further showcase the generalization performance of FL-Aler on these four datasets, we conducted cross-experiments on them.

Table 6. Comparison Results of FL-AIer with the State-of-the-Art Methods

Datasets	Method	A@1	A@2	A@3	A@5	MAR	Datasets	Method	A@1	A@2	A@3	A@5	MAR
Data1	Brandón	17.23	26.78	36.12	46.95	23.87	Data3	Brandón	35.12	51.98	59.76	67.54	27.45
	iSQUAD	22.85	31.02	40.12	45.78	13.04		iSQUAD	32.14	56.72	64.03	75.89	3.72
	DT	54.11	62.34	66.22	67.19	17.89		DT	39.85	45.12	46.34	57.68	12.02
	Gradient Boosting	64.23	73.10	75.49	80.97	4.01		Gradient Boosting	45.32	68.45	74.89	78.22	3.28
	Random Forest	74.56	88.72	90.85	94.67	2.07		Random Forest	60.87	76.35	81.23	86.45	2.17
	SVM	32.45	40.12	43.87	52.23	8.13		SVM	27.96	47.45	59.89	72.58	4.30
	RandomWalk@Metric	6.12	9.23	13.45	25.98	10.89		RandomWalk@Metric	11.42	17.89	33.56	48.12	7.96
	RandomWalk@FI	15.67	18.45	21.12	19.89	32.78		RandomWalk@FI	24.56	35.78	41.23	51.89	6.97
	DéjàVu	76.34	89.45	92.56	97.23	1.78		DéjàVu	62.34	81.45	89.78	93.67	3.56
	FL-AIer	82.45	92.67	94.78	97.89	1.34		FL-AIer	63.78	86.45	91.67	95.67	1.56
Data2	Brandón	14.85	20.32	23.87	29.76	46.89	Data4	Brandón	9.12	8.54	9.31	12.68	302.17
	iSQUAD	7.92	11.34	14.68	25.03	29.87		iSQUAD	22.35	31.56	29.74	32.01	260.43
	DT	54.72	61.23	62.87	66.19	31.45		DT	63.89	71.56	74.03	72.58	129.21
	Gradient Boosting	71.12	76.45	75.89	79.23	9.01		Gradient Boosting	51.23	60.14	58.67	64.32	34.21
	Random Forest	72.78	78.12	80.34	84.67	9.12		Random Forest	86.14	85.78	90.12	90.67	12.89
	SVM	0.23	1.67	1.78	11.89	24.45		SVM	1.12	6.89	10.23	15.21	91.78
	RandomWalk@Metric	10.01	19.78	27.89	32.14	14.67		RandomWalk@Metric	3.23	6.45	8.23	12.78	105.34
	RandomWalk@FI	20.78	22.34	29.12	39.78	29.56		RandomWalk@FI	0.23	0.12	0.56	1.89	444.78
	DéjàVu	65.34	70.12	74.23	78.45	5.45		DéjàVu	76.34	83.56	91.23	93.78	3.34
	FL-AIer	83.89	86.23	88.67	89.45	3.01		FL-AIer	88.23	91.67	95.12	97.23	2.45

Boldface indicates the optimal experimental results obtained by various methods under the corresponding indicators.

Methodology. In Data1, we used A, B, and C as the training and validation sets, and D as the test set. In Data2, we used B, C, and D as the training and validation sets, and A as the test set. In Data3, we used A, C, and D as the training and validation sets, and B as the test set. In Data4, we used A, B, and D as the training and validation sets, and C as the test set. To ensure the stability of the experiments, we repeated each experiment 10 times and took the average value.

Results. Table 6 presents the results of fault localization of various methods across four datasets in cross-experiments. The experimental results demonstrate that FL-AIer outperforms state-of-the-art methods in terms of generalization performance, as evidenced by metrics such as A@1, A@2, A@3, A@5, and MAR. For instance, compared to state-of-the-art methods, on Data1, FL-AIer showed improvements in A@1 by 378.53%, 260.83%, 52.37%, 28.37%, 10.58%, 154.08%, 1,247.22%, 426.16%, and 8.00%. On Data2, FL-AIer showed improvements in A@1 by 465.92%, 959.22%, 53.31%, 17.96%, 15.27%, 36,373.91%, 738.06%, 303.71%, and 28.39%. On Data3, the improvements in A@1 were 81.61%, 98.44%, 60.05%, 40.73%, 4.78%, 128.11%, 458.49%, 159.69%, and 2.31%. On Data4, the increases were 867.43%, 294.77%, 38.10%, 72.22%, 2.43%, 77,776.79%, 2,631.58%, 38,260.87%, and 15.58%. FL-AIer also significantly surpassed state-of-the-art methods in other measurement metrics. These data further demonstrate that FL-AIer has better generalization performance.

Answering RQ6: Based on the above experimental results, it can be seen that FL-AIer performs well in fault localization across four datasets in cross-validation, indicating that the FL-AIer model has good generalization performance.

7 Threats to Validity

7.1 Internal Validity

Firstly, one potential threat to internal validity is the replicability and accuracy of state-of-the-art methods. When replicating models, we ensure the use of the same parameter settings as the original article, relying entirely on the provided code and environment. Secondly, another potential threat to internal validity is the assessment of our method's performance. To mitigate potential errors in the code, we have validated it. Upon acceptance of this study, we will openly release our code and dataset, allowing other researchers to replicate and extend this work.

7.2 Construct Validity

The primary threats to construct validity lie in hyperparameters and evaluation metrics. We tuned the hyperparameters of FL-AIer and the state-of-the-arts methods through grid search, following existing work. Additionally, we employed widely used evaluation metrics. In the future, we plan to further optimize the performance of FL-AIer models through experimental analysis to determine the optimal parameters.

7.3 External Validity

The primary threat to external validity lies in the selection of study subjects. We evaluated FL-AIer on four systems, which may not represent all online service systems. However, we believe our approach is sufficiently generalizable for two reasons. Firstly, these four systems come from different companies and have entirely different architectures. Specifically, FL-AIer is even applicable to traditional software systems (e.g., Oracle databases in *sys_C*). Secondly, our approach is applicable to other online service systems because its input data (e.g., metrics, historical failures, and relationships between components) is common, and FL-AIer is scalable. In the future, we will mitigate this threat by evaluating FL-AIer on more systems.

8 Conclusions and Future Work

In this article, we introduced a model named FL-AIer that utilizes GATs to encode the spatial and temporal features of faulty units. Additionally, it employs a multi-head attention mechanism and a weighted KL divergence loss function to address the challenges of complex dependency relations among faulty units and the imbalance in fault knowledge. Our experiments on fault datasets from real systems of two major banks and an Internet service provider, as well as an open source dataset, show that FL-AIer performs well on metrics such as A@1, A@2, A@3, A@5, and MAR. Compared to state-of-the-art methods, FL-AIer improved A@1 by 5.82%, 10.77%, 4.20%, and 15.56%, respectively. The experimental results demonstrate that FL-AIer effectively addresses the dependency issues of fault characteristics and the imbalanced fault knowledge distribution, significantly enhancing the accuracy and reducing localization time. However, our model also has clear limitations. Firstly, for unknown fault types, it has unpredictable fault indicators, which can cause problems when using historical fault localization. Secondly, using all fault indicators in each fault unit would lead to an excessive number of fault indicators, resulting in high input dimensions and difficulty in training. In future work, we plan to further research computing the similarity between unknown faults and historical faults to improve fault localization performance for unknown faults. Additionally, we

intend to filter fault indicators in the preprocessing stage, focusing more on fault types that have a decisive impact on faults.

References

- [1] Junjie Chen, Xiaoting He, Qingwei Lin, Yong Xu, Hongyu Zhang, Dan Hao, Feng Gao, Zhangwei Xu, Yingnong Dang, and Dongmei Zhang. 2019. An empirical investigation of incident triage for online service systems. In *2019 IEEE/ACM 41st International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, 111–120.
- [2] Junjie Chen, Xiaoting He, Qingwei Lin, Hongyu Zhang, Dan Hao, Feng Gao, Zhangwei Xu, Yingnong Dang, and Dongmei Zhang. 2019. Continuous incident triage for large-scale online service systems. In *2019 34th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 364–375.
- [3] Zhuangbin Chen, Jinyang Liu, Yuxin Su, Hongyu Zhang, Xiao Ling, Yongqiang Yang, and Michael R. Lyu. 2022. Adaptive performance anomaly detection for online service systems via pattern sketching. In *the 44th International Conference on Software Engineering (ICSE '22)*. ACM, New York, NY, 61–72.
- [4] Xu Zhang, Yong Xu, Qingwei Lin, Bo Qiao, Hongyu Zhang, Yingnong Dang, Chunyu Xie, Xinsheng Yang, Qian Cheng, Ze Li, et al. 2019. Robust log-based anomaly detection on unstable log data. In *2019 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE '19)*. ACM, New York, NY, 807–817.
- [5] Zhengran Zeng, Yuqun Zhang, Yong Xu, Minghua Ma, Bo Qiao, Wentao Zou, Qingjun Chen, Meng Zhang, Xu Zhang, Hongyu Zhang, et al. 2023. TraceArk: Towards actionable performance anomaly alerting for online service systems. In *2023 IEEE/ACM 45th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, 258–269.
- [6] Leonardo Mariani, Mauro Pezzè, Oliviero Riganelli, and Rui Xin. 2020. Predicting failures in multi-tier distributed systems. *Journal of Systems and Software* 161 (2020), 110464.
- [7] Cuong Pham, Long Wang, Byung Chul Tak, Salman Baset, Chunqiang Tang, Zbigniew Kalbarczyk, and Ravishankar K. Iyer. 2017. Failure diagnosis for distributed systems using targeted fault injection. *IEEE Transactions on Parallel and Distributed Systems* 28, 2 (2017), 503–516.
- [8] Vipshop. 2024. Retrieved from <https://baijiahao.baidu.com/s?id=1768011934535401914&wfr=spider&for=pc>
- [9] DIDI. 2024. Retrieved from <https://baijiahao.baidu.com/s?id=1783859161793432092&wfr=spider&for=pc>
- [10] Qingyang Yu, Nengwen Zhao, Mingjie Li, Zeyan Li, Honglin Wang, Wenchi Zhang, Kaixin Sui, and Dan Pei. 2024. A survey on intelligent management of alerts and incidents in IT services. *Journal of Network and Computer Applications* 224 (2024), 103842.
- [11] Peter Bodik, Moises Goldszmidt, Armando Fox, Dawn B. Woodard, and Hans Andersen. 2010. Fingerprinting the datacenter: Automated classification of performance crises. In *5th European Conference on Computer Systems (EuroSys '10)*. ACM, New York, NY, 111–124.
- [12] Xiang Zhou, Xin Peng, Tao Xie, Jun Sun, Chao Ji, Dewei Liu, Qilin Xiang, and Chuan He. 2019. Latent error prediction and fault localization for microservice applications by learning from system Trace logs. In *2019 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE '19)*. ACM, New York, NY, 683–694.
- [13] Zeyan Li, Nengwen Zhao, Mingjie Li, Xianglin Lu, Lixin Wang, Dongdong Chang, Xiaohui Nie, Li Cao, Wenchi Zhang, Kaixin Sui, et al. 2022. Actionable and interpretable fault localization for recurring failures in online service systems. In *30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE '22)*. ACM, New York, NY, 996–1008.
- [14] Yueshen Xu, Zhibo Qiu, Honghao Gao, Xinkui Zhao, Lu Wang, and Rui Li. 2024. Heterogeneous data-driven failure diagnosis for microservice-based industrial clouds towards consumer digital ecosystems. *IEEE Transactions on Consumer Electronics* 70, 1 (2024), 2027–2037.
- [15] Jing Ren, Feng Xia, Ivan Lee, Azadeh Noori Hoshyar, and Charu Aggarwal. 2023. Graph learning for anomaly analytics: Algorithms, applications, and challenges. *ACM Transactions on Intelligent Systems and Technology* 14, 2 (Feb. 2023), 1–29.
- [16] Yupeng Zhu, Yanxiang Chen, Zuxing Zhao, Xueliang Liu, and Jinlin Guo. 2023. Local self-attention-based hybrid multiple instance learning for partial spoof speech detection. *ACM Transactions on Intelligent Systems and Technology* 14, 5 (Oct. 2023), 1–18.
- [17] Xiang Zhou, Xin Peng, Tao Xie, Jun Sun, Chao Ji, Wenhai Li, and Dan Ding. 2021. Fault analysis and debugging of microservice systems: Industrial survey, benchmark system, and empirical study. *IEEE Transactions on Software Engineering* 47, 2 (2021), 243–260.
- [18] FL-Hunter. 2024. Our Replication Package. Retrieved from <https://github.com/Alegkria/FL-Hunter>

- [19] Pradeep Dogga, Karthik Narasimhan, Anirudh Sivaraman, Shiv Saini, George Varghese, and Ravi Netravali. 2022. Revelio: ML-generated debugging queries for finding root causes in distributed systems. In *Proceedings of Machine Learning and Systems*. D. Marculescu, Y. Chi, and C. Wu (Eds.), Vol. 4, 601–622.
- [20] I. Lee and R. K. Iyer. 2000. Diagnosing rediscovered software problems using symptoms. *IEEE Transactions on Software Engineering* 26, 2 (2000), 113–127.
- [21] Álvaro Brandón, Marc Solé, Alberto Huélamo, David Solans, María S. Pérez, and Victor Muntés-Mulero. 2020. Graph-based root cause analysis for service-oriented and microservice architectures. *Journal of Systems and Software* 159 (2020), 110432.
- [22] Pradeep Dogga, Karthik Narasimhan, Anirudh Sivaraman, Shiv Saini, George Varghese, and Ravi Netravali. 2022. Revelio: ML-generated debugging queries for finding root causes in distributed systems. *Proceedings of Machine Learning and Systems* 4 (2022), 601–622.
- [23] Ping Liu, Yu Chen, Xiaohui Nie, Jing Zhu, Shenglin Zhang, Kaixin Sui, Ming Zhang, and Dan Pei. 2019. FluxRank: A widely-deployable framework to automatically localizing root cause machines for software service failure mitigation. In *2019 IEEE 30th International Symposium on Software Reliability Engineering (ISSRE)*, 35–46.
- [24] Minghua Ma, Zheng Yin, Shenglin Zhang, Sheng Wang, Christopher Zheng, Xinhao Jiang, Hanwen Hu, Cheng Luo, Yilin Li, Nengjun Qiu, et al. 2020. Diagnosing root causes of intermittent slow queries in cloud databases. *Proceedings of the VLDB Endowment* 13, 8 (Apr. 2020), 1176–1189.
- [25] David Bau, Bolei Zhou, Aditya Khosla, Aude Oliva, and Antonio Torralba. 2017. Network dissection: Quantifying interpretability of deep visual representations. In *2017 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 3319–3327.
- [26] Vincenzo Di Cicco, Donatella Firmani, Nick Koudas, Paolo Merialdo, and Divesh Srivastava. 2019. Interpreting deep learning models for entity resolution: An experience report using LIME. In *2nd International Workshop on Exploiting Artificial Intelligence Techniques for Data Management (aiDM '19)*. ACM, New York, NY, Article 8, 4 pages.
- [27] Matthew D. Zeiler and Rob Fergus. 2014. Visualizing and understanding convolutional networks. In *Computer Vision (ECCV '14)*. David Fleet, Tomas Pajdla, Bernt Schiele, and Tinne Tuytelaars (Eds.), Springer International Publishing, Cham, 818–833.
- [28] Myunghwan Kim, Roshan Sumbaly, and Sam Shah. 2013. Root cause detection in a service-oriented architecture. *ACM SIGMETRICS Performance Evaluation Review* 41, 1 (Jun. 2013), 93–104.
- [29] Ruyue Xin, Peng Chen, and Zhiming Zhao. 2023. CausalRCA: Causal inference based precise fine-grained root cause localization for microservice applications. *Journal of Systems and Software* 203 (2023), 111724.
- [30] Shilin He, Qingwei Lin, Jian-Guang Lou, Hongyu Zhang, Michael R. Lyu, and Dongmei Zhang. 2018. Identifying impactful service system problems via log analysis. In *2018 26th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE '18)*. ACM, New York, NY, 60–70.
- [31] Qingwei Lin, Hongyu Zhang, Jian-Guang Lou, Yu Zhang, and Xuwei Chen. 2016. Log clustering based problem identification for online service systems. In *38th International Conference on Software Engineering Companion (ICSE '16)*. ACM, New York, NY, 102–111.
- [32] Animesh Nandi, Atri Mandal, Shubham Atreja, Gargi B. Dasgupta, and Subhrajit Bhattacharya. 2016. Anomaly detection using program control flow graph mining from execution logs. In *22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD '16)*. ACM, New York, NY, 215–224.
- [33] Zeyan Li, Junjie Chen, Rui Jiao, Nengwen Zhao, Zhijun Wang, Shuwei Zhang, Yanjun Wu, Long Jiang, Lei Qin Yan, Zikai Wang, et al. 2021. Practical root cause localization for microservice systems via trace analysis. In *2021 IEEE/ACM 29th International Symposium on Quality of Service (IWQOS)*, 1–10.
- [34] Guangba Yu, Pengfei Chen, Hongyang Chen, Zijie Guan, Zicheng Huang, Linxiao Jing, Tianjun Weng, Xinmeng Sun, and Xiaoyun Li. 2021. MicroRank: End-to-end latency issue localization with extended spectrum analysis in microservice environments. In *the Web Conference 2021 (WWW '21)*. ACM, New York, NY, 3087–3098.
- [35] Yang Cai, Biao Han, Jie Li, Na Zhao, and Jinshu Su. 2021. ModelCoder: A fault model based automatic root cause localization framework for microservice systems. In *2021 IEEE/ACM 29th International Symposium on Quality of Service (IWQOS)*, 1–6.
- [36] Xiaofeng Guo, Xin Peng, Hanzhang Wang, Wanxue Li, Huai Jiang, Dan Ding, Tao Xie, and Liangfei Su. 2020. Graph-based trace analysis for microservice architecture understanding and problem diagnosis. In *28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE '20)*. ACM, New York, NY, 1387–1397.
- [37] Guangba Yu, Pengfei Chen, Yufeng Li, Hongyang Chen, Xiaoyun Li, and Zibin Zheng. 2023. Nezha: Interpretable fine-grained root causes analysis for microservices on multi-modal observability data. In *31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. ACM, New York, NY, 553–565.

- [38] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. 2016. *Deep Learning*. MIT Press. Retrieved from <http://www.deeplearningbook.org>
- [39] Fadi Dornaika. 2023. One-step multi-view clustering with consensus graph and data representation convolution. *ACM Transactions on Intelligent Systems and Technology* 15, 1 (Dec. 2023), 1–24.
- [40] shayXu. 2024. GELU. Retrieved from <https://zhuanlan.zhihu.com/p/302394523?>
- [41] Will Hamilton, Zitao Ying, and Jure Leskovec. 2017. Inductive representation learning on large graphs. In *Advances in Neural Information Processing Systems*. I. Guyon, U. Von Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett (Eds.), Vol. 30, Curran Associates, Inc. Retrieved from https://proceedings.neurips.cc/paper_files/paper/2017/file/5dd9db5e033da9c6fb5ba83c7a7e9bea9-Paper.pdf
- [42] Shaohan Chen, Chuanhou Gao, and Ping Zhang. 2022. Incorporation of data-mined knowledge into black-box SVM for interpretability. *ACM Transactions on Intelligent Systems and Technology* 14, 1 (Nov. 2022).
- [43] Li Wu, Johan Tordsson, Jasmin Bogatinovski, Erik Elmroth, and Odej Kao. 2021. MicroDiag: Fine-grained performance diagnosis for microservice systems. In *2021 IEEE/ACM International Workshop on Cloud Intelligence (CloudIntelligence)*, 31–36.
- [44] Minjie Wang, Da Zheng, Zihao Ye, Quan Gan, Mufei Li, Xiang Song, Jinjing Zhou, Chao Ma, Lingfan Yu, Yu Gai, et al. 2019. Deep graph library: A graph-centric, highly-performant package for graph neural networks. arXiv:1909.01315. Retrieved from <https://arxiv.org/abs/1909.01315>
- [45] Edmundo de Souza e Silva and H. Richard Gail. 1989. Calculating availability and performability measures of repairable computer systems using randomization. *Journal of the ACM* 36, 1 (Jan. 1989), 171–193.
- [46] Mike Chen, Emre Kiciman, Eugene Fratkin, Armando Fox, and Eric Brewer. 2002. Pinpoint: Problem determination in large, dynamic Internet services. In *Proceedings International Conference on Dependable Systems and Networks*, 595–604.
- [47] James A. Jones, Mary Jean Harrold, and John T. Stasko. 2002. Visualization of test information to assist fault localization. In *Proceedings of the 24th International Conference on Software Engineering (ICSE '02)*, 467–477.
- [48] Lee Naish, Huajie Lee, and Kotagiri Ramamohanarao. 2011. A model for spectra-based software diagnosis. *ACM Transactions on Software Engineering and Methodology* (2011).
- [49] W. Eric Wong, Vidroha Debroy, Ruizhi Gao, and Yihao Li. 2014. The DStar method for effective software fault localization. *IEEE Transactions on Reliability* 63, 1 (2014), 290–308.
- [50] Huijiang Liu, Shuirou Jiang, Xuexin Qi, Yang Qu, Hui Li, Tingting Li, Cheng Guo, and Shikai Guo. 2024. Detect software vulnerabilities with weight biases via graph neural networks. *Expert Systems with Applications* 238 (2024), 121764.
- [51] Yaqin Zhou, Shangqing Liu, Jingkai Siow, Xiaoning Du, and Yang Liu. 2019. Devign: Effective vulnerability identification by learning comprehensive program semantics via graph neural networks. In *Advances in Neural Information Processing Systems*. H. Wallach, H. Larochelle, A. Beygelzimer, F. d'Alché-Buc, E. Fox, and R. Garnett (Eds.), Vol. 32, Curran Associates, Inc.
- [52] Guohao Li, Matthias Muller, Ali Thabet, and Bernard Ghanem. 2019. DeepGCNs: Can GCNs go as deep as CNNs? In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 9267–9276.

Received 3 May 2024; revised 10 November 2024; accepted 29 December 2024